

- This page has great information about memory barriers on Linux: <https://www.mjmwired.net/kernel/Documentation/memory-barriers.txt#305>

4.10 SIMD/Vector Processing

In Section 4.1.4, we introduced a form of parallelism known as *single instruction multiple data* (SIMD). This refers to the ability of most modern microprocessors to perform a mathematical operation on multiple data items *in parallel*, using a single machine instruction. In this section, we'll explore SIMD techniques in some detail, and conclude the chapter with a brief discussion of how SIMD and multithreading are combined into a form of parallelism known *single instruction multiple thread* (SMT), which forms the basis of all modern GPUs.

Intel first introduced its MMX¹¹ instruction set with their Pentium line of CPUs in 1994. These instructions permitted SIMD calculations to be performed on eight 8-bit integers, four 16-bit integers, or two 32-bit integers packed into special 64-bit MMX registers. Intel followed this up with various revisions of an extended instruction set called *streaming SIMD extensions*, or SSE, the first version of which appeared in the Pentium III processor.

The SSE instruction set utilizes 128-bit registers that can contain integer or IEEE floating-point data. The SSE mode most commonly used by game engines is called *packed 32-bit floating-point mode*. In this mode, four 32-bit `float` values are packed into a single 128-bit register. An operation such as addition or multiplication can thus be performed on four pairs of `floats` in parallel by taking two of these 128-bit registers as its inputs. Intel has since introduced various upgrades to the SSE instruction set, named SSE2, SSE3, SSSE3 and SSE4. In 2007, AMD introduced its own variants named XOP, FMA4 and CVT16.

In 2011, Intel introduced a new, wider SIMD register file and accompanying instruction set named *advanced vector extensions* (AVX). AVX registers are 256 bits wide, permitting a single instruction to operate on pairs of up to eight 32-bit floating-point operands in parallel. The AVX2 instruction set is an extension to AVX. Some Intel CPUs now support AVX-512, an extension to AVX permitting 16 32-bit floats to be packed into a 512-bit register.

¹¹Officially, MMX is a meaningless initialism trademarked by Intel. Unofficially, developers consider it to stand for “multimedia extensions” or “matrix math extensions.”

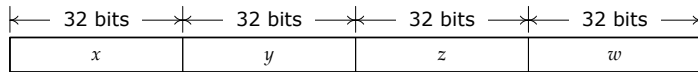


Figure 4.38. The four components of an SSE register in 32-bit floating-point mode.

4.10.1 The SSE Instruction Set and Its Registers

The SSE instruction set includes a wide variety of operations, with many variants for operating on differently-sized data elements within SSE registers. For the purposes of the present discussion, however, we'll stick to the relatively small subset of instructions that deal with packed 32-bit floating-point data. These instructions are denoted with a `ps` suffix, indicating that we're dealing with *packed* data (`p`), and that each element is a *single*-precision `float` (`s`). However most of the upcoming discussions extend intuitively into AVX's 256- and 512-bit modes; see <https://software.intel.com/en-us/articles/introduction-to-intel-advanced-vector-extensions> for an overview of AVX.

The SSE registers are named `XMMi`, where i is an integer between 0 and 15 (e.g., `XMM0`, `XMM1`, and so on). In packed 32-bit floating-point mode, each 128-bit `XMMi` register contains four 32-bit floats. In AVX, the registers are 256 bits wide and are named `YMMi`; in AVX-512, they are 512 bits in width and are named `ZMMi`.

In this chapter, we'll often refer to the individual floats within an SSE register as $[x \ y \ z \ w]$, just as they would be when doing vector/matrix math in homogeneous coordinates on paper (see Figure 4.38). It doesn't usually matter what you call the elements of an SSE register, as long as you're consistent about how you *interpret* each element. The most general approach is to think of an SSE vector `r` as containing the elements $[r_0 \ r_1 \ r_2 \ r_3]$. Most SSE documentation uses this convention, although some documentation uses a $[w \ x \ y \ z]$ convention, so be careful out there!

4.10.1.1 The `__m128` Data Type

In order for SSE instructions to perform arithmetic on packed floating-point data, that data must reside in one of the `XMMi` registers. For long-term storage, packed floating-point data can of course reside in memory, but it must be transferred from RAM into an SSE register prior to being used for any calculations, and the results subsequently transferred back to RAM.

To make working with SSE and AVX data easier, C and C++ compilers provide special data types that represent packed arrays of `float`s. The `__m128` type encapsulates a packed array of four `float`s for use with the SSE intrinsics. (The `__m256` and `__m512` types likewise represent packed arrays of eight

and 16 floats, respectively, for use with AVX intrinsics.)

The `__m128` data type and its kin can be used to declare global variables, automatic variables, function arguments and return types, and even class and structure members. Declaring automatic variables and function arguments to be of type `__m128` often results in the compiler treating those values as direct proxies for SSE registers. But using the `__m128` type to declare global variables, structure/class members, and sometimes local variables results in the data being stored as a 16-byte aligned array of `float` in memory. Using a memory-based `__m128` variable in an SSE calculation will cause the compiler to implicitly emit instructions for loading the data from memory into an SSE register prior to performing the calculation on it, and likewise to emit instructions for storing the results of the calculation back into the memory that “backs” each such variable. As such, it’s a good idea to check the disassembly to make sure that you’re not doing unnecessary loads and stores of SSE registers when using the `__m128` type (and its AVX relatives).

4.10.1.2 Alignment of SSE Data

Whenever data that is intended for use in an `XMMi` register is stored in memory, it must be 16-byte (128-bit) aligned. (Likewise, data intended for use with AVX’s 256-bit `YMMi` registers must be 32-byte (256-bit) aligned, and data for use with the 512-bit `ZMMi` registers must be 64-byte (512-bit) aligned.)

The compiler ensures that global and local variables of type `__m128` are aligned automatically. It also pads `struct` and `class` members so that any `__m128` members are aligned properly *relative to* the start of the object, and ensures that the alignment of the entire `struct` or `class` is equal to the worst-case alignment of its members. This means that declaring a global or local variable instance of a `struct` or `class` that includes at least one `__m128` member will be 16-byte aligned by the compiler automatically.

However, all *dynamically allocated* instances of such a `struct` or `class` need to be aligned manually. Likewise, any array of `float` that you intend to use with SSE instructions must be properly aligned; you can ensure this via the C++11 `alignas` specifier. See Section 6.2.1.3 for more information on aligned memory allocations.

4.10.1.3 SSE Compiler Intrinsics

We could work directly with the SSE and AVX assembly language instructions, perhaps using our compiler’s inline assembly syntax. However, writing code like this is not only non-portable, it’s also a big pain in the butt! To make life easier, modern compilers provide *intrinsics*—special syntax that looks and

behaves like a regular C function, but is actually boiled down to inline assembly code by the compiler. Many intrinsics translate into a single assembly language instruction, although some are *macros* that translate into a sequence of instructions.

In order to use SSE and AVX intrinsics, your .cpp file must `#include <xmmintrin.h>` in Visual Studio, or `<x86intrin.h>` when compiling with Clang or gcc.

4.10.1.4 Some Useful SSE Intrinsics

There are a lot of SSE intrinsics, but for the purposes of our discussion here we can start with only five of them:

- `__m128 _mm_set_ps(float w, float z, float y, float x);`
This intrinsic initializes an `__m128` variable with the four floating-point values provided.
- `__m128 _mm_load_ps(const float* pData);`
This intrinsic loads four `float`s from a C-style array into an `__m128` variable. The input array must be 16-byte aligned.
- `void _mm_store_ps(float* pData, __m128 v);`
This intrinsic stores the contents of an `__m128` variable into a C-style array of four `float`s, which must be 16-byte aligned.
- `__m128 _mm_add_ps(__m128 a, __m128 b);`
This intrinsic adds the four pairs of `float`s contained in the variables `a` and `b` in parallel and returns the result.
- `__m128 _mm_mul_ps(__m128 a, __m128 b);`
This intrinsic multiplies the four pairs of `float`s contained in the variables `a` and `b` in parallel and returns the result.

You may have noticed that the arguments `x`, `y`, `z` and `w` are passed to the `_mm_set_ps()` function in reverse order. This strange convention probably arises from the fact that Intel CPUs are *little-endian*. Just as a single floating-point value with the bit pattern `0x12345678` would be stored in memory as the bytes `0x78`, `0x56`, `0x34`, `0x12` in order of increasing addresses, so too are the contents of an SSE register stored in memory in an order that's opposite to the order those components actually appear within the register. In other words, not only are the four bytes comprising each float in an SSE register stored in little-endian order, but so too are the four floats themselves. All of this is just a question of naming convention: There's really no "most-significant"

or “least-significant” float within an SSE register. So we can either treat the in-memory order as the “correct” order and consider `_mm_set_ps()` to be “backwards,” or we can treat the arguments of `_mm_set_ps()` as being in the “correct” order and think of all in-memory vectors as being “backwards.” We’ll stick with the former convention, since it means that we’ll be able to read off our vectors more naturally: A homogeneous vector v consisting of members (v_x, v_y, v_z, v_w) would be stored in a C/C++ array as `float v[] = { vx, vy, vz, vw }`, but would be passed to `_mm_set_ps()` as `w, z, y, x`.

Here’s a small code snippet that loads two four-element floating-point vectors, adds them, and prints the results.

```
#include <xmmintrin.h>

void TestAddSSE()
{
    alignas(16) float A[4];
    alignas(16) float B[4] = { 2.0f, 4.0f, 6.0f, 8.0f };

    // set a = (1, 2, 3, 4) from literal values, and
    // load b = (2, 4, 6, 8) from a floating-point array
    // just to illustrate the two ways of doing this
    // (remember that _mm_set_ps() is backwards!)
    __m128 a = _mm_set_ps(4.0f, 3.0f, 2.0f, 1.0f);
    __m128 b = _mm_load_ps(&B[0]);

    // add the vectors
    __m128 r = _mm_add_ps(a, b);

    // store '__m128 a' into a float array for printing
    _mm_store_ps(&A[0], a);

    // store result into a float array for printing
    alignas(16) float R[4];
    _mm_store_ps(&R[0], r);

    // inspect results
    printf("a = %.1f %.1f %.1f %.1f\n",
           A[0], A[1], A[2], A[3]);
    printf("b = %.1f %.1f %.1f %.1f\n",
           B[0], B[1], B[2], B[3]);
    printf("r = %.1f %.1f %.1f %.1f\n",
           R[0], R[1], R[2], R[3]);
}
```

4.10.1.5 AltiVec vector Types

As a quick aside, the GNU C/C++ compiler *gcc* (used to compile code for the PS3, for example) provides support for the PowerPC's *AltiVec* instruction set, which provides support for SIMD operations, much as SSE does on Intel processors. 128-bit vector types can be declared like regular C/C++ types, but they are preceded by the keyword `vector`. For example, a SIMD variable containing four floats would be declared `vector float`. *gcc* also provides a means of writing *literal* SIMD values into your source code. For example, you can initialize a `vector float` with a value like this:

```
vector float v = (vector float)(-1.0f, 2.0f, 0.5f, 1.0f);
```

The corresponding Visual Studio code is a tad more clunky:

```
// use compiler intrinsic to load "literal" value
// (remember _mm_set_ps() is backwards!)
__m128 v = _mm_set_ps(1.0f, 0.5f, 2.0f, -1.0f);
```

We won't cover AltiVec explicitly in this chapter, but once you understand SSE it'll be very easy to learn.

4.10.2 Using SSE to Vectorize a Loop

SIMD offers the potential to speed up certain kinds of calculations by a factor of four, because a single machine language instruction can perform an arithmetic operation on four floating-point values in parallel. Let's see how this can be done using SSE intrinsics.

First, consider a simple loop that adds two arrays of floats pairwise, storing each result into an output array:

```
void AddArrays_ref(int count,
                  float* results,
                  const float* dataA,
                  const float* dataB,
                  {
    for (int i = 0; i < count; ++i)
    {
        results[i] = dataA[i] + dataB[i];
    }
}
```

We can speed up this loop significantly using SSE intrinsics, like this:

```

void AddArrays_sse(int count,
                  float* results,
                  const float* dataA,
                  const float* dataB)
{
    // NOTE: the caller needs to ensure that the size of
    // all 3 arrays are equal, and a multiple of four!
    assert(count % 4 == 0);

    for (int i = 0; i < count; i += 4)
    {
        __m128 a = _mm_load_ps(&dataA[i]);
        __m128 b = _mm_load_ps(&dataB[i]);
        __m128 r = _mm_add_ps(a, b);
        _mm_store_ps(&results[i], r);
    }
}

```

In this version, we loop over the values four at a time. We load blocks of four floats into SSE registers, add them in parallel, and store the results into a corresponding block of four floats within the result array. This is called *vectorizing* our loop. (In this example, we’re assuming that the sizes of the three arrays are equal, and that the size is a multiple of four. The caller is responsible for *padding* the arrays with one, two or three dummy values each, as necessary, in order to meet this requirement.)

Vectorization can lead to a significant speed improvement. This particular example isn’t quite four times faster, due to the overhead of having to load the values in groups of four and store the results on each iteration; but when I measured these functions running on very large arrays of floats, the non-vectorized loop took roughly 3.8 times as long to do its work as the vectorized loop.

4.10.3 Vectorized Dot Products

Let’s apply vectorization to a somewhat more interesting task: Calculating dot products. Given two arrays of four-element vectors, the goal is to calculate their dot products pairwise, and store the results into an output array of floats.

Here’s a reference implementation without using SSE. In this implementation, each contiguous block of four floats within the `a[]` and `b[]` input arrays is interpreted as one homogeneous four-element vector.

```

void DotArrays_ref(int count,
                  float r[],
                  const float a[],

```

```

        const float b[]))
{
    for (int i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const int j = i * 4;

        r[i] = a[j+0]*b[j+0] // ax*bx
              + a[j+1]*b[j+1] // ay*by
              + a[j+2]*b[j+2] // az*bz
              + a[j+3]*b[j+3]; // aw*bw
    }
}

```

4.10.3.1 A First Attempt (That's Slow)

Here's a first attempt at using SSE intrinsics for this task:

```

void DotArrays_sse_horizontal(int count,
                              float r[],
                              const float a[],
                              const float b[])
{
    for (int i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const int j = i * 4;

        __m128 va = _mm_load_ps(&a[j]); // ax,ay,az,aw
        __m128 vb = _mm_load_ps(&b[j]); // bx,by,bz,bw

        __m128 v0 = _mm_mul_ps(va, vb);

        // add across the register...
        __m128 v1 = _mm_hadd_ps(v0, v0);
        // (v0w+v0z, v0y+v0x, v0w+v0z, v0y+v0x)
        __m128 vr = _mm_hadd_ps(v1, v1);
        // (v0w+v0z+v0y+v0x, v0w+v0z+v0y+v0x,
        //   v0w+v0z+v0y+v0x, v0w+v0z+v0y+v0x)

        _mm_store_ss(&r[i], vr); // extract vr.x as a float
    }
}

```

This implementation required a new instruction: `_mm_hadd_ps()` (*horizontal*

add). This intrinsic operates on a single input register (x, y, z, w) and calculates two sums: $s = x + y$ and $t = z + w$. It stores these two sums into the four slots of the destination register as (t, s, t, s) . Performing this operation twice allows us to calculate the sum $d = x + y + z + w$. This is called *adding across a register*.

Adding across a register is not usually a good idea because it's a very *slow* operation. Profiling the `DotArrays_sse()` implementation shows that it actually takes a little bit *more* time than the reference implementation. Using SSE here has actually slowed us down!¹²

4.10.3.2 A Better Approach

The key to realizing the power of SIMD parallelism for dot products is to figure out a way to avoid having to add across a register. This can be done, but we'll have to *transpose* our input vectors first. By storing them in transposed order, we can calculate our dot product in just the same way that we calculated it when using floats: We multiply the x components, then add that result to the product of the y components, then add that result to the product of the z components, and finally add that result to the product of the w components. Here's what the code looks like:

```
void DotArrays_sse(int count,
                  float r[],
                  const float a[],
                  const float b[])
{
    for (int i = 0; i < count; i += 4)
    {
        __m128 vaX = _mm_load_ps(&a[(i+0)*4]); // a[0,4,8,12]
        __m128 vaY = _mm_load_ps(&a[(i+1)*4]); // a[1,5,9,13]
        __m128 vaZ = _mm_load_ps(&a[(i+2)*4]); // a[2,6,10,14]
        __m128 vaW = _mm_load_ps(&a[(i+3)*4]); // a[3,7,11,15]

        __m128 vbX = _mm_load_ps(&b[(i+0)*4]); // b[0,4,8,12]
        __m128 vbY = _mm_load_ps(&b[(i+1)*4]); // b[1,5,9,13]
        __m128 vbZ = _mm_load_ps(&b[(i+2)*4]); // b[2,6,10,14]
        __m128 vbW = _mm_load_ps(&b[(i+3)*4]); // b[3,7,11,15]

        __m128 result;
        result = _mm_mul_ps(vaX, vbX);
        result = _mm_add_ps(result, _mm_mul_ps(vaY, vbY));
        result = _mm_add_ps(result, _mm_mul_ps(vaZ, vbZ));
    }
}
```

¹²With SSE4, Intel introduced the intrinsic `_mm_dp_ps()` (and the corresponding `dpps` instruction) which calculates a dot product with somewhat lower latency than the version involving two invocations of `_mm_hadd_ps()`. But all horizontal adds are very expensive, and should be avoided wherever possible.

```

        result = _mm_add_ps(result, _mm_mul_ps(vaW, vbW));
    }
    _mm_store_ps(&r[i], result);
}

```

The MADD Instruction

It's interesting to note that a multiply followed by an add is such a common operation that it has its own name—*madd*. Some CPUs provide a single SIMD instruction for performing a *madd* operation. For example, the PowerPC AltiVec intrinsic `vec_madd()` performs this operation. So in AltiVec, the guts of our `DotArrays()` function could be made just a little bit simpler:

```

vector float result = vec_mul(vaX, vbX);
result = vec_madd(vaY, vbY, result);
result = vec_madd(vaZ, vbZ, result);
result = vec_madd(vaW, vbW, result);

```

4.10.3.3 Transpose as We Go

The above implementation *assumes* that the input data has already been transposed by the caller. In other words, the `a[]` array is assumed to contain the components $\{a_0, a_4, a_8, a_{12}, a_1, a_5, a_9, a_{13}, \dots\}$ and likewise for the `b[]` array. If we want to operate on input data that's in the same format as it was for the reference implementation, we'll have to do the transposition within our function. Here's how:

```

void DotArrays_sse_transpose(int count,
                             float r[],
                             const float a[],
                             const float b[])
{
    for (int i = 0; i < count; i += 4)
    {
        __m128 vaX = _mm_load_ps(&a[(i+0)*4]); // a[0,1,2,3]
        __m128 vaY = _mm_load_ps(&a[(i+1)*4]); // a[4,5,6,7]
        __m128 vaZ = _mm_load_ps(&a[(i+2)*4]); // a[8,9,10,11]
        __m128 vaW = _mm_load_ps(&a[(i+3)*4]); // a[12,13,14,15]

        __m128 vbX = _mm_load_ps(&b[(i+0)*4]); // b[0,1,2,3]
        __m128 vbY = _mm_load_ps(&b[(i+1)*4]); // b[4,5,6,7]
        __m128 vbZ = _mm_load_ps(&b[(i+2)*4]); // b[8,9,10,11]
        __m128 vbW = _mm_load_ps(&b[(i+3)*4]); // b[12,13,14,15]
    }
}

```

```

    _MM_TRANSPOSE4_PS(vaX, vaY, vaZ, vaW);
                                // vaX = a[0,4,8,12]
                                // vaY = a[1,5,9,13]
                                // ...
    _MM_TRANSPOSE4_PS(vbX, vbY, vbZ, vbW);
                                // vbX = b[0,4,8,12]
                                // vbY = b[1,5,9,13]
                                // ...

    __m128 result;
    result = _mm_mul_ps(vaX, vbX);
    result = _mm_add_ps(result, _mm_mul_ps(vaY, vbY));
    result = _mm_add_ps(result, _mm_mul_ps(vaZ, vbZ));
    result = _mm_add_ps(result, _mm_mul_ps(vaW, vbW));

    _mm_store_ps(&r[i], result);
}
}

```

Those two calls to `_MM_TRANSPOSE()` are actually invocations of a somewhat complex macro that uses *shuffle* instructions to move the components of the four input registers around. Thankfully shuffling isn't a particularly expensive operation, so transposing our vectors as we calculate the dot products doesn't introduce too much overhead. Profiling all three implementations of `DotArrays()` shows that our final version (the one that transposes the vectors as it goes) is roughly 3.5 times faster than the reference implementation.

4.10.3.4 Shuffle and Transpose

For the curious reader, here's what the `_MM_TRANSPOSE()` macro looks like:

```

#define _MM_TRANSPOSE4_PS(row0, row1, row2, row3) \
{ __m128 tmp3, tmp2, tmp1, tmp0; \
 \
    tmp0 = _mm_shuffle_ps((row0), (row1), 0x44); \
    tmp2 = _mm_shuffle_ps((row0), (row1), 0xEE); \
    tmp1 = _mm_shuffle_ps((row2), (row3), 0x44); \
    tmp3 = _mm_shuffle_ps((row2), (row3), 0xEE); \
 \
    (row0) = _mm_shuffle_ps(tmp0, tmp1, 0x88); \
    (row1) = _mm_shuffle_ps(tmp0, tmp1, 0xDD); \
    (row2) = _mm_shuffle_ps(tmp2, tmp3, 0x88); \
    (row3) = _mm_shuffle_ps(tmp2, tmp3, 0xDD); }

```

Those crazy hexadecimal numbers are bit-packed four-element fields called

shuffle masks. They tell the `_mm_shuffle()` intrinsic how exactly to shuffle the components. These bit-packed fields are a common source of confusion, possibly because of the naming conventions used in most documentation. But it's actually pretty simple: A shuffle mask is constructed out of four integers, each of which represents one of the components of an SSE register (and hence can have a value between 0 and 3).

```
#define SHUFMASK(p,q,r,s) \
    (p | (q<<2) | (r<<4) | (s<<6))
```

Passing two SSE registers `a` and `b` along with a shuffle mask to the `_mm_shuffle_ps()` intrinsic results in the specified components of `a` and `b` appearing in the output register `r` as follows:

```
__m128 a = ...;
__m128 b = ...;
__m128 r = _mm_shuffle_ps(a, b,
                          SHUFMASK(p,q,r,s));
// r == ( a[p], a[q], b[r], b[s] )
```

4.10.4 Vector-Matrix Multiplication with SSE

Now that we understand how to perform a dot product, we can multiply a four-element vector with a 4×4 matrix. To do it, we simply need to perform four dot products between the input vector and each of the four rows of the input matrix.

We'll start by defining a `Mat44` class that encapsulates four SSE vectors, representing its four rows. We'll use a union so that we can easily access the individual members of the matrix as floats. (This works because instances of our `Mat44` class will always reside in memory, never directly in SSE registers.)

```
union Mat44
{
    float c[4][4]; // components
    __m128 row[4]; // rows
};
```

The function to multiply a vector and a matrix looks like this:

```
__m128 MulVecMat_sse(const __m128& v, const Mat44& M)
{
    // first transpose v
    __m128 vX = _mm_shuffle_ps(v, v, 0x00); // (vx,vx,vx,vx)
```



```

__m128 vY = _mm_shuffle_ps(v, v, 0x55); // (vy,vy,vy,vy)
__m128 vZ = _mm_shuffle_ps(v, v, 0xAA); // (vz,vz,vz,vz)
__m128 vW = _mm_shuffle_ps(v, v, 0xFF); // (vw,vw,vw,vw)

__m128 r =      _mm_mul_ps(vX, M.row[0]);
r = _mm_add_ps(r, _mm_mul_ps(vY, M.row[1]));
r = _mm_add_ps(r, _mm_mul_ps(vZ, M.row[2]));
r = _mm_add_ps(r, _mm_mul_ps(vW, M.row[3]));
return r;
}

```

The shuffles are used to replicate *one* component of **v** (either v_x , v_y , v_z or v_w) across *all four* lanes of an SSE register. This has the effect of transposing **v** prior to performing the dot product with the rows of **M**, which are already transposed. (Remember that vector-matrix multiplication normally involves taking dot products between an input vector and the *columns* of the matrix. Here, we’re transposing **v** into four SSE registers, and then doing our math, component-wise, with the *rows* of the matrix.)

4.10.5 Matrix-Matrix Multiplication with SSE

Multiplying two 4×4 matrices with SSE intrinsics is trivial once we have a function to multiply a vector and a matrix. Here’s what the code looks like:

```

void MulMatMat_sse(Mat44& R, const Mat44& A, const Mat44& B)
{
    R.row[0] = MulVecMat_sse(A.row[0], B);
    R.row[1] = MulVecMat_sse(A.row[1], B);
    R.row[2] = MulVecMat_sse(A.row[2], B);
    R.row[3] = MulVecMat_sse(A.row[3], B);
}

```

4.10.6 Generalized Vectorization

Because an SSE register contains four floating-point values, it’s tempting to think of it as a natural “fit” for the components of a four-element homogeneous vector **v**, and to think that the best use of SSE is for doing 3D vector math. However, this is a very limiting way of thinking about SIMD parallelism.

Most “batched” operations, in which a single computation is performed repeatedly on a large dataset, can be vectorized using SIMD parallelism. If you think about it, the components of a SIMD register really function like parallel “lanes” in which arbitrary processing can be performed. Working with `float`

variables gives us a single lane, but working with 128-bit (four-element) SIMD variables allows us to do that same calculation in parallel across four lanes—in other words, we can perform our calculations four at a time. Working with 256-bit AVX registers gives us eight lanes, allowing us to perform our calculations eight at a time. And AVX-512 gives us 16 lanes, letting us do 16 calculations at a time.

The easiest way to write *vectorized* code is to start out by writing it as a single-lane algorithm (just using `floats`). Once it works, we can convert it to operate N elements at a time, using SIMD registers that have an N -lane capacity. We’ve already seen this process in action: In Section 4.10.3, we first wrote a loop that performed a large batch of dot products one at a time, and then we converted to use SSE so that we could perform those dot products four at a time.

One nice side-effect of vectorizing your code in this way is that you can reap the benefits of wider SIMD hardware with little adjustment to your code. On a machine with only SSE support, you perform four operations per iteration of your loop; on a machine that supports AVX, you simply change it to do eight operations per iteration; and on an AVX-512 system, you can do 16 operations per iteration.

Interestingly, most optimizing compilers can *vectorize* some kinds of single-lane loops automatically. In fact, when writing the above examples, it took some doing to force the compiler *not* to vectorize my single-lane code, so that I could compare its performance to my SIMD implementation! Once again, it’s always a good idea to look at the disassembly when writing optimized code—you may discover the compiler is doing more (or less) than you thought!

4.10.7 Vector Predication

Let’s take a look at another (totally contrived) example. This example will reinforce the ideas of generalized vectorization, but it will also serve to illustrate another useful technique: *vector predication*.

Imagine that we needed to take the square roots of a large array of `floats`. We’d start out by writing it as a single-lane loop, like this:

```
#include <cmath>

void SqrtArray_ref(float* __restrict__ r,
                  const float* __restrict__ a,
                  int count)
{
    for (unsigned i = 0; i < count; ++i)
    {
```

```

        if (a[i] >= 0.0f)
            r[i] = std::sqrt(a[i]);
        else
            r[i] = 0.0f;
    }
}

```

Next, let's convert this loop into SSE, performing four square roots at a time:

```

#include <xmmintrin.h>

void SqrtArray_sse_broken(float* __restrict__ r,
                          const float* __restrict__ a,
                          int count)
{
    assert(count % 4 == 0);
    __m128 vz = _mm_set1_ps(0.0f); // all zeros

    for (int i = 0; i < count; i += 4)
    {
        __m128 va = _mm_load_ps(a + i);

        __m128 vr;
        if (_mm_cmpge_ps(va, vz)) // ???
            vr = _mm_sqrt_ps(va);
        else
            vr = vz;

        _mm_store_ps(r + i, vr);
    }
}

```

This seems simple enough: We simply stride through the input array four floats at a time, load groups of four floats into an SSE register, and then perform four parallel square roots with `_mm_sqrt_ps()`.

However, there's one small gotcha in this loop. We need to check whether the input values are greater than or equal to zero, because the square root of a negative number is imaginary (and will therefore produce QNaN). The intrinsic `_mm_cmpge_ps()` compares the values of two SSE registers, component-wise, to see if they are greater than or equal to a vector of values supplied by the caller. However, this function doesn't return a `bool`. How can it? We're comparing four values to four other values, so some of them may pass the test while others fail it. That means we can't just do an `if` check against the results of `_mm_cmpge_ps()`.¹³

¹³The `if` check in the single-lane reference implementation also prevents the compiler from automatically vectorizing the loop.

Does this spell doom for our vectorized implementation? Thankfully not. All SSE comparison intrinsics like `_mm_cmpge_ps()` produce a four-component result, stored in an SSE register. But instead of containing four floating-point values, the result consists of four 32-bit masks. Each mask contains all binary 1s (0xFFFFFFFFU) if the corresponding component in the input register passed the test, and all binary 0s (0x0U) if that component failed the test.

We can use the results of an SSE comparison intrinsic by applying it as a bit mask in order to *select* between one of two possible results. In our example, when the input value passes the test (is greater than or equal to zero), we want to select the square root of that input value; when it fails the test (is negative), we want to select a value of zero. This is called *predication*, and because we're applying it to SIMD vectors, it's called *vector predication*.

In Section 4.2.6.2, we saw how to do predication with floating-point values, using bitwise AND, OR and NOT operators. Here's an excerpt from that example:

```
// ...

// select quotient when mask is all ones, or default
// value d when mask is all zeros (NOTE: this won't
// work as written -- you'd need to use a union to
// interpret the floats as unsigned for masking)
const float result = (q & mask) | (d & ~mask);
return result;
}
```

It's no different here, we just need to use SSE versions of these bitwise operators:

```
#include <xmmintrin.h>

void SqrtArray_sse(float* __restrict__ r,
                  const float* __restrict__ a,
                  int count)
{
    assert(count % 4 == 0);
    __m128 vz = _mm_set1_ps(0.0f);

    for (int i = 0; i < count; i += 4)
    {
        __m128 va = _mm_load_ps(a + i);
```

```

    // always do the quotient, but it may end
    // up producing QNaN in some or all lanes
    __m128 vq = _mm_sqrt_ps(va);

    // now select between vq and vz, depending
    // on whether the input was greater than
    // or equal to zero or not
    __m128 mask = _mm_cmpge_ps(va, zero);

    // (vq & mask) | (vz & ~mask)
    __m128 qmask = _mm_and_ps(mask, vq);
    __m128 znotmask = _mm_andnot_ps(mask, vz);
    __m128 vr = _mm_or_ps(qmask, znotmask);

    _mm_store_ps(r + i, vr);
}
}

```

It's convenient and commonplace to encapsulate this *vector predication* operation in a function, which is typically called *vector select*. PowerPC's AltiVec ISA provides an intrinsic called `vec_sel()` for this purpose. It works like this:

```

// pseudocode illustrating how AltiVec's vec_sel() intrinsic
// works
vector float vec_sel(vector float falseVec,
                      vector float trueVec,
                      vector bool mask)
{
    vector float r;
    for (each lane i)
    {
        if (mask[i] == 0)
            r[i] = falseVec[i];
        else
            r[i] = trueVec[i];
    }
    return r;
}

```

SSE2 provided no vector select instruction, but thankfully one was introduced in SSE4—it is emitted by the intrinsic `_mm_blendv_ps()`.

Let's take a look at how we might implement a vector select operation ourselves. We can write it like this:

```

__m128 __mm_select_ps(const __m128 a,
                      const __m128 b,
                      const __m128 mask)
{
    // (b & mask) | (a & ~mask)
    __m128 bmask = __mm_and_ps(mask, b);
    __m128 anotmask = __mm_andnot_ps(mask, a);
    return __mm_or_ps(bmask, anotmask);
}

```

Or if we're feeling brave, we can accomplish the same thing with exclusive OR:

```

__m128 __mm_select_ps(const __m128 a,
                      const __m128 b,
                      const __m128 mask)
{
    // (((a ^ b) & mask) ^ a)
    __m128 diff = __mm_xor_ps(a, b);
    return __mm_xor_ps(a, __mm_and_ps(mask, diff));
}

```

See if you can figure out how this works. Here are two hints: The exclusive OR operator calculates the *bitwise difference* between two values. Two XORs in a row leave the input value unchanged ($((a \oplus b) \oplus b == a)$).

4.11 Introduction to GPGPU Programming

We said in the preceding section that most optimizing compilers can *vectorize* some code automatically, if the target CPU includes a SIMD vector processing unit, and if the source code meets certain requirements (such as not involving complex branching). Vectorization is also one of the pillars of *general-purpose GPU* programming (GPGPU). In this section, we'll take a brief introductory look at how a GPU differs from a CPU in terms of its hardware architecture, and how the concepts of SIMD parallelism and vectorization enable programmers to write *compute shaders* that are capable of processing large amounts of data in parallel on a GPU.

4.11.1 Data-Parallel Computations

A GPU is a specialized coprocessor designed specifically to accelerate those computations that involve a high degree of *data parallelism*. It does so by combining SIMD parallelism (vectorized ALUs) with MIMD parallelism (by employing a form of preemptive multithreading). NVIDIA coined the term *single instruction multiple thread* (SIMT) to describe this SIMD/MIMD hybrid design,

although the design is not unique to NVIDIA GPUs—although the specifics of GPU designs vary from vendor to vendor and from product line to product line in significant ways, all GPUs employ the general principles of SIMT parallelism in their designs.

GPUs are designed specifically to perform *data-parallel* computations on very large datasets. In order for a computational task to be well-suited to execution on a GPU, the computations performed on any one element of the dataset must be *independent* of the results of computations on other elements. In other words, it must be possible to process the elements in any order.

The simple examples of SIMD vectorization that we presented starting in Section 4.10.3 are all examples of data-parallel computations. Recall this function, which processes two potentially very large arrays of input vectors and produces an output array containing the scalar dot products of those vectors:

```
void DotArrays_ref(unsigned count,
                  float r[],
                  const float a[],
                  const float b[])
{
    for (unsigned i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const unsigned j = i * 4;

        r[i] = a[j+0]*b[j+0] // ax*bx
              + a[j+1]*b[j+1] // ay*by
              + a[j+2]*b[j+2] // az*bz
              + a[j+3]*b[j+3]; // aw*bw
    }
}
```

The computation performed by each iteration of this loop is independent of the computations performed by the other iterations. That means that we are free to perform the computations in any order we see fit. Moreover, this property is what allows us to apply SSE or AVX intrinsics to vectorize the loop—instead of performing the computations one-at-a-time, we can use SIMD parallelism to perform four, eight or 16 computations simultaneously, thereby reducing the iteration count by a factor of four, eight or 16, respectively.

Now imagine taking SIMD parallelization to an extreme. What if we had a SIMD VPU with 1024 lanes? In that case, we could divide the total number of iterations by 1024—and when the input arrays contain 1024 elements or fewer, we could literally execute the entire loop in a single iteration!

This is, roughly speaking, what a GPU does. However, it doesn't use SIMDs that are *actually* 1024 lanes wide each. A GPU's *SIMD units* are typically eight or 16 lanes wide, but they process workloads in batches of 32 or 64 elements at a time. What's more, a GPU contains *many* such SIMD units. So a large workload can be dispatched across these SIMD units in parallel, resulting in the GPU being capable of processing literally thousands of data elements in parallel.

Data-parallel computations are just what the doctor ordered when applying a pixel shader (also known as a fragment shader) to millions of pixels, or a vertex shader to hundreds of thousands or even millions of 3D mesh vertices, every frame at 30 or 60 FPS. But modern GPUs expose their computational power to programmers, allowing us to write general-purpose *compute shaders*. As long as the computations we wish to perform on a large dataset have the property of being largely independent of one another, they can probably be executed by a GPU more efficiently than they could be on a CPU.

4.11.2 Compute Kernels

In Section 4.10.3, we saw that in order to vectorize a loop like the one in the `DotArrays_ref()` function, we must rewrite the code. The vectorized version of the function makes use of SSE or AVX intrinsics; our *scalar* data types are replaced by *vector* types such as SSE's `__m128` or AltiVec's `vector float`; and the loop is hard-coded to iterate four, eight or 16 elements at a time.

When writing a GPGPU compute shader, we take a different tack. Instead of hard-coding the loop to operate in fixed-size batches, we leave the loop as a "single-lane" computation using scalar data types. Then, we extract the body of the loop into a separate function known as a *kernel*. Here's what the example above would look like as a kernel:

```
void DotKernel(unsigned i,
               float r[],
               const float a[],
               const float b[])
{
    // treat each block of four floats as a
    // single four-element vector
    const unsigned j = i * 4;

    r[i] = a[j+0]*b[j+0] // ax*bx
          + a[j+1]*b[j+1] // ay*by
          + a[j+2]*b[j+2] // az*bz
          + a[j+3]*b[j+3]; // aw*bw
}
```



```

void DotArrays_gpgpu1(unsigned count,
                      float r[],
                      const float a[],
                      const float b[])
{
    for (unsigned i = 0; i < count; ++i)
    {
        DotKernel(i, r, a, b);
    }
}

```

The `DotKernel()` function is now in a form that's suitable for conversion into a *compute shader*. It processes just one element of the input data, and produces a single output element. This is analogous to how a pixel/fragment shader receives a single input pixel/fragment color and transforms it into a single output color, or to how a vertex shader receives a single input vertex and produces a single output vertex. The GPU effectively runs the `for` loop for us, calling our kernel function once for each element of our dataset.

GPGPU compute kernels are typically written in a special *shading language* which can be compiled into machine code that's understandable by the GPU. Shading languages are usually very close to C in syntax, so converting a C or C++ loop into a GPU compute kernel isn't usually a particularly difficult undertaking. Some examples of shading languages include DirectX's HLSL (high-level shader language), OpenGL's GLSL, NVIDIA's Cg and CUDA C languages, and OpenCL.

Some shading languages require you to move your kernel code into special source files, separate from your C++ application code. OpenCL and CUDA C, however, are extensions to the C++ language itself. As such, they permit programmers to write compute kernels as regular C/C++ functions, with only minor syntactic adjustments, and to invoke those kernels on the GPU with relatively simple syntax.

As a concrete example, here's our `DotKernel()` function written in CUDA C:

```

__global__ void DotKernel_CUDA(int count,
                               float* r,
                               const float* a,
                               const float* b)
{
    // CUDA provides a magic "thread index" to each
    // invocation of the kernel -- this serves as
    // our loop index i

```

```

size_t i = threadIdx.x;

// make sure the index is valid
if (i < count)
{
    // treat each block of four floats as a
    // single four-element vector
    const unsigned j = i * 4;

    r[i] = a[j+0]*b[j+0] // ax*bx
          + a[j+1]*b[j+1] // ay*by
          + a[j+2]*b[j+2] // az*bz
          + a[j+3]*b[j+3]; // aw*bw
}
}

```

You'll notice that the loop index `i` is taken from a variable called `threadIdx` within the kernel function itself. The thread index is a "magic" input provided by the compiler, in much the same way that the `this` pointer "magically" points to the current instance within a C++ class member function. We'll talk more about the thread index in the next section.

4.11.3 Executing a Kernel

Given that we've written a compute kernel, let's see how to execute it on the GPU. The details differ from shading language to shading language, but the key concepts are roughly equivalent. For example, here's how we'd kick off our compute kernel in CUDA C:

```

void DotArrays_gpgpu2(unsigned count,
                      float r[],
                      const float a[],
                      const float b[])
{
    // allocate "managed" buffers that are visible
    // to both CPU and GPU
    int *cr, *ca, *cb;
    cudaMallocManaged(&cr, count * sizeof(float));
    cudaMallocManaged(&ca, count * sizeof(float) * 4);
    cudaMallocManaged(&cb, count * sizeof(float) * 4);

    // transfer the data into GPU-visible memory
    memcpy(ca, a, count * sizeof(float) * 4);
    memcpy(cb, b, count * sizeof(float) * 4);
}

```

```

// run the kernel on the GPU
DotKernel_CUDA <<<1, count>>> (cr, ca, cb, count);

// wait for GPU to finish
cudaDeviceSynchronize();

// return results and clean up
memcpy(r, cr, count * sizeof(float));
cudaFree(cr);
cudaFree(ca);
cudaFree(cb);
}

```

A bit of set-up code is required to allocate the input and output buffers as “managed” memory that is visible to both CPU and GPU, and to copy the input data into them. The CUDA-specific triple angled brackets notation (<<<G,N>>>) then executes the compute kernel on the GPU, by submitting a request to the driver. The call to `cudaDeviceSynchronize()` forces the CPU to wait until the GPU has done its work (in much the same way that `pthread_join()` forces one thread to wait for the completion of another). Finally, we free the GPU-visible data buffers.

Let’s have a closer look at the <<<G,N>>> angled bracket notation. The second argument N within the angled brackets allows us to specify the *dimensions* of our input data. This corresponds to the number of *iterations* of our loop that we want the GPU to perform. It can actually be a one-, two- or three-dimensional quantity, allowing us to process one-, two- or three-dimensional input arrays. However, just like in C/C++, a multidimensional array is really just a one-dimensional array that’s indexed in a special way. For example, in C/C++ a two-dimensional array access written like `[row][column]` is really equivalent to a one-dimensional array access `[row*numColumns + column]`. The same principle applies to multidimensional GPU buffers.

The first argument G in the angled brackets tells the driver how many *thread groups* (known as *thread blocks* in NVIDIA terminology) to use when running this compute kernel. A compute job with a single thread group is constrained to run on a single *compute unit* on the GPU. A compute unit is essentially a *core* within the GPU—a hardware component that is capable of executing an instruction stream. Passing a larger number for G allows the driver to divide the workload up across more than one compute unit.

4.11.4 GPU Threads and Thread Groups

The G argument tells the GPU driver into how many *thread groups* to divide our work. As you might expect, a thread group is comprised of some number

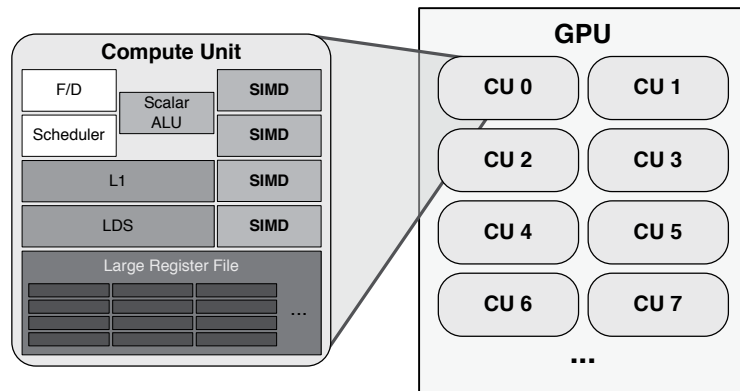


Figure 4.39. A typical GPU is comprised of multiple compute units (CU), each of which acts like a stripped down CPU core. A CU contains an instruction fetch/decode unit, an L1 cache, possibly a block of local data storage RAM (LDS), a scalar ALU, and a number of SIMD units for executing vectorized code.

of *GPU threads*. But what exactly does the term “thread” mean in the context of a GPU?

Every GPU kernel is compiled into an *instruction stream* consisting of a sequence of GPU machine language instructions, in much the same way that a C/C++ function is compiled down into a stream of CPU instructions. So a GPU “thread” is equivalent to a CPU “thread,” in the sense that both kinds of threads represent a stream of machine language instructions that can be executed by one or more cores. However, a GPU executes its threads in a manner that is somewhat different from the way in which a CPU executes its threads. As a result, the term “thread” has a subtly different meaning when applied to GPU compute kernels than it has when applied to running programs on a CPU.

In order to understand this difference in terminology, let’s take a brief glimpse at the architecture of a GPU. We said in Section 4.11.1 that a GPU is comprised of multiple *compute units*, each of which contains some number of *SIMD units*. We can think of a compute unit like a stripped down CPU core: It contains an instruction fetch/decode unit, possibly some memory caches, a regular “scalar” ALU, and usually somewhere in the neighborhood of four SIMD units, which serve much the same function as the vector processing unit (VPU) in a SIMD-enabled CPU. This architecture is illustrated in Figure 4.39.

The SIMD units in a CU have different lane widths on different GPUs, but for the sake of this discussion let’s assume we’re working on an AMD

Radeon™ Graphics Core Next (GCN) architecture, in which the SIMDs are 16 lanes wide. The CU is not capable of speculative or out-of-order execution—it simply reads a stream of instructions and executes them one by one, using the SIMD units to apply each instruction¹⁴ to 16 elements of input data in parallel.

To execute a compute kernel on a CU, the driver first subdivides the input data buffers into blocks consisting of 64 elements each. For each of these 64-element blocks of data, the compute kernel is invoked on one CU. Such an invocation is called a *wavefront* (also known as a *warp* in NVIDIA speak). When executing a wavefront, the CU fetches and decodes the instructions of the kernel one by one. Each instruction is applied to 16 data elements in *lock step* using a SIMD unit. Internally, the SIMD unit consists of a four-*stage* pipeline, so it takes four clock cycles to complete. So rather than allow the stages of this pipeline to sit idle for three clock cycles out of every four, the CU applies the same instruction three more times, to three more blocks of 16 data elements. This is why a wavefront consists of 64 data elements, even though the SIMDs in the CU are only 16 lanes wide.

Because of this somewhat peculiar way in which a CU executes an instruction stream, the term “*GPU thread*” actually refers to a single SIMD *lane*. You can think of a GPU thread, therefore, as a single iteration of the original non-vectorized loop that we started with, before we converted its body into a compute kernel. Alternatively, you can think of a GPU thread as a single invocation of the kernel function, operating on a single input datum and producing a single output datum. The fact that the GPU actually runs multiple GPU threads in parallel (i.e., the fact that it really runs the kernel once per wavefront, but processes 64 data elements at a time) is just an implementation detail. By insulating the programmer from having to think about the details of how the computation is vectorized on any particular GPU, compute kernels (and graphics shaders as well) can be written in a portable manner.

4.II.4.1 From SIMD to SIMT

The term *single instruction multiple thread* (SIMT) was introduced to underscore the fact that a GPU doesn’t just use SIMD parallelism—it also uses a form of preemptive multithreading to time-slice between wavefronts. Let’s take a brief look at why this is done.

A SIMD unit runs a wavefront by applying each instruction in the shader program to 64 data elements at a time, essentially in lock-step. (We can ignore the fact that the wavefront is processed in subgroups of 16 elements each for the purposes of the present discussion.) However, any one instruction in the

¹⁴The compute units on a GPU do contain a scalar ALU and therefore can perform some instructions in “single lane” fashion, operating on a single input datum at a time.

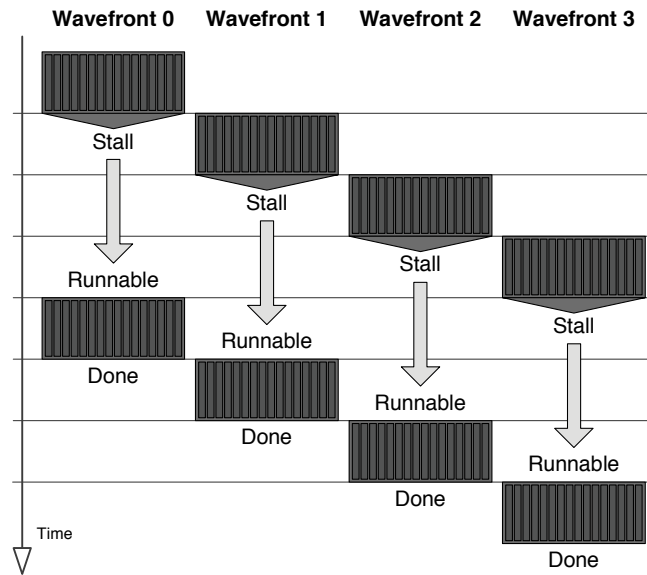


Figure 4.40. Whenever one wavefront stalls, for example due to memory access latency, the SIMD unit context switches to another wavefront to fill the delay slot.

program might end up having to access memory, and that introduces a large stall while the SIMD unit waits for the memory controller to respond.

To fill these large delay slots, a SIMD unit time-slices between multiple wavefronts (taken from a single shader program, or potentially from many unrelated shader programs). Whenever one wavefront stalls, the SIMD unit context switches to another wavefront, thereby keeping the unit busy (as long as there are runnable wavefronts to switch to). This strategy is illustrated in Figure 4.40.

As you might well imagine, the SIMD units in a GPU need to perform context switches at a very high frequency. During a context switch on a CPU, the state of the outgoing thread's registers are saved to memory so they won't be lost, and then the state of the incoming thread's registers are read from memory into the CPU's registers so that it can continue executing where it left off. However, on a GPU it would take too much time to save the state of each wavefront's SIMD registers every time a context switch occurred.

To eliminate the cost of saving registers during context switches, each SIMD unit contains a very large register file. The number of *physical* registers in this register file is many times larger than the number of *logical* registers available to any one wavefront (typically on the order of ten times larger).

This means that the contents of the logical registers for up to ten wavefronts can be maintained at all times in these physical registers. And that in turn implies that context switches can be performed between wavefronts without saving or restoring any registers whatsoever.

4.II.5 Further Reading

Obviously GPGPU programming is a huge topic, and as always we've only just scratched the surface in this book. For more information on GPGPU and graphics shader programming, check out the following online tutorials and resources:

- Introduction to CUDA programming: <https://developer.nvidia.com/how-to-cuda-c-cpp>
- OpenCL learning resources: <https://developer.nvidia.com/opencl>
- HLSL programming guide and reference manual: [https://msdn.microsoft.com/en-us/library/bb509561\(v=VS.85\).aspx](https://msdn.microsoft.com/en-us/library/bb509561(v=VS.85).aspx)
- Introduction to the OpenGL shading language: https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language
- AMD Radeon™ GCN architecture whitepaper: https://www.amd.com/Documents/GCN_Architecture_whitepaper.pdf



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

5

3D Math for Games

A game is a mathematical model of a virtual world simulated in real time on a computer of some kind. Therefore, mathematics pervades everything we do in the game industry. Game programmers make use of virtually all branches of mathematics, from trigonometry to algebra to statistics to calculus. However, by far the most prevalent kind of mathematics you'll be doing as a game programmer is 3D vector and matrix math (i.e., 3D *linear algebra*).

Even this one branch of mathematics is very broad and very deep, so we cannot hope to cover it in any great depth in a single chapter. Instead, I will attempt to provide an overview of the mathematical tools needed by a typical game programmer. Along the way, I'll offer some tips and tricks, which should help you keep all of the rather confusing concepts and rules straight in your head. For an excellent in-depth coverage of 3D math for games, I highly recommend Eric Lengyel's book [32] on the topic. Chapter 3 of Christer Ericson's book [14] on real-time collision detection is also an excellent resource.

5.1 Solving 3D Problems in 2D

Many of the mathematical operations we're going to learn about in the following chapter work equally well in 2D and 3D. This is very good news, because